

Detecção de Objetos Aéreos com YOLOv8

João Pedro Nunes Oliveira

Abstract—This work introduces a computer vision system designed for the automatic recognition of aerial entities such as unidentified flying objects (UFOs), airplanes, helicopters, birds, and drones. The solution is based on the YOLOv8 object detection architecture, selected for its optimal trade-off between precision and computational performance. To train a lightweight variant of the model (YOLOv8n), a custom dataset containing 225 training images and 20 validation images was utilized, along with data augmentation strategies to improve generalization. The resulting model can detect and classify aerial objects in real-time video streams, laying the groundwork for future developments in aerial monitoring, anomaly detection, and environmental observation.

I. INTRODUÇÃO

NOS últimos anos, os avanços em visão computacional e inteligência artificial têm viabilizado sistemas capazes de interpretar imagens e vídeos com alta precisão. Essas tecnologias são aplicadas em áreas como segurança, monitoramento ambiental, controle aéreo e pesquisa científica.

Um dos principais desafios é a identificação automática de objetos aéreos — aviões, helicópteros, drones, pássaros e até OVNI — essencial para vigilância, análise de tráfego e investigação de fenômenos anômalos, especialmente com o aumento de dispositivos no espaço aéreo.

Este projeto propõe um sistema de detecção automática baseado no modelo YOLOv8, focado em eficiência e operação em tempo real. Utilizando um conjunto de dados customizado com diversas classes de objetos aéreos, o modelo foi treinado para realizar detecção precisa em vídeos reais.

O YOLOv8 foi escolhido por sua leveza e capacidade de funcionar em dispositivos com recursos limitados. A proposta demonstra que, mesmo com poucos dados e hardware simples, é possível desenvolver uma solução eficaz para análise automatizada do espaço aéreo.

28 de Outubro, 2025

II. ETAPAS DO DESENVOLVIMENTO

O desenvolvimento do projeto seguiu uma sequência lógica de etapas, abrangendo desde a coleta das imagens até o teste final do modelo treinado. As etapas principais estão descritas abaixo.

A. Coleta das Imagens

As imagens foram coletadas com o auxílio da extensão Imageye, disponível para o navegador Google Chrome, que permite o download em massa de imagens a partir de resultados de pesquisa. Essa ferramenta facilitou a obtenção de um número representativo de imagens contendo diferentes tipos de objetos aéreos, como OVNI, aviões, helicópteros, pássaros e drones.

J. P. N. Oliveira is with the Federal University of Viçosa (UFV), Viçosa-MG, Brazil. E-mail: jpedro.open@gmail.com.

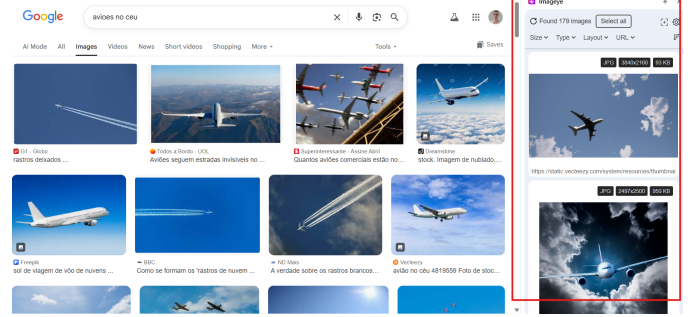


Fig. 1. Representação do funcionamento do Imageye.

B. Criação das classes

Cinco classes de interesse foram estabelecidas — OVNI, avião, helicóptero, pássaro e drone — representando as categorias que o modelo foi projetado para identificar e distinguir.

C. Anotação das Imagens

As imagens foram anotadas manualmente com a ferramenta online MakeSense.ai, que oferece uma interface intuitiva para marcação de regiões de interesse e suporta diversos formatos de exportação. Para cada imagem, foi gerado um arquivo de texto individual (.txt) contendo as informações no formato compatível com o modelo YOLO.

Cada arquivo .txt possui uma linha por objeto identificado, seguindo a estrutura:

```
<classe> <x_centro> <y_centro> <largura> <altura>
```

Os valores de posição e dimensão são expressos como coordenadas normalizadas (entre 0 e 1), representando o centro do objeto e o tamanho relativo da caixa delimitadora (bounding box) em relação à resolução da imagem.

Esse processo assegura que o modelo YOLO interprete corretamente as regiões anotadas durante o treinamento, associando as coordenadas espaciais aos respectivos rótulos de classe — OVNI, avião, helicóptero, pássaro e drone.

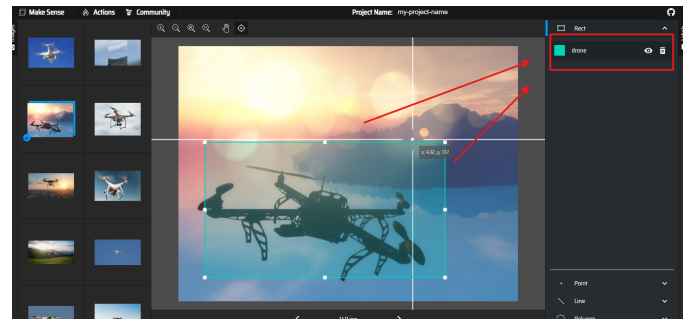


Fig. 2. Representação do funcionamento da MakeSenseAI.

D. Divisão do Conjunto de Dados

As imagens foram organizadas em dois subconjuntos: 225 para treinamento (train) e 20 para validação (val). Por se tratar de um projeto experimental e demonstrativo, optou-se por utilizar um volume reduzido de dados, com o objetivo de minimizar o tempo de processamento e ainda assim permitir uma avaliação funcional do desempenho do modelo.

No entanto, essa limitação pode impactar a capacidade de generalização do sistema, especialmente em relação a classes menos representadas. Com poucos exemplos disponíveis, é possível que algumas categorias — como OVNI's ou drones — não sejam corretamente detectadas.

E. Configuração do arquivo data.yaml

Foi criado o arquivo data.yaml, que define os caminhos para os diretórios de imagens de treinamento e validação, além de listar as classes que o modelo deve reconhecer.



Fig. 3. Arquivo data.yaml.

F. Treinamento Customizado

O modelo yolov8n.pt, previamente treinado em um conjunto de dados genérico, foi adaptado ao novo dataset por meio de fine-tuning, utilizando o seguinte comando:

```
yolo detect train \
  model=yolov8n.pt \
  data=dataset/data.yaml \
  epochs=50 \
  imgsz=640 \
  augment=true
```

O treinamento foi realizado ao longo de 50 épocas, com imagens de 640×640 pixels — uma escolha estratégica para reduzir o tempo de processamento e garantir agilidade na execução do experimento. Por se tratar de um projeto demonstrativo, optou-se por imagens de menor resolução, visando otimizar o desempenho computacional sem comprometer a capacidade de aprendizado do modelo.

O parâmetro augment=true ativa automaticamente técnicas de aumento de dados, como rotação, redimensionamento e variações de brilho e cor, que contribuem para uma melhor generalização em diferentes cenários. Durante os testes, observou-se que essas técnicas ajudaram o modelo a manter estabilidade nas detecções ao longo dos vídeos, mesmo diante de variações sutis de posição, iluminação ou escala dos objetos.

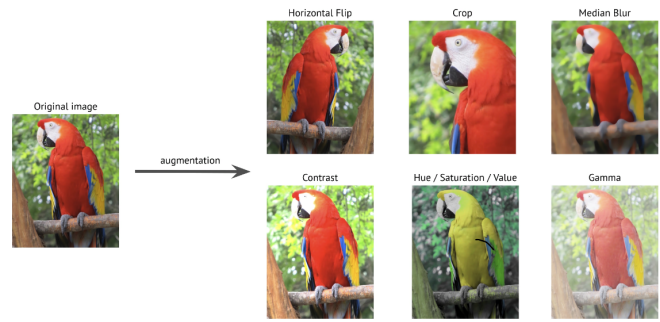


Fig. 4. Representação do fluxo de processamento das imagens realizado pelo modelo YOLO, conforme descrito em sua documentação oficial.

G. Avaliação do modelo

Entre as métricas geradas, as curvas de F1 e precisão em função da confiança se destacaram por evidenciar o desempenho do modelo em cada classe. Observou-se maior dificuldade na detecção das classes drone e ovni, que apresentaram baixos valores e instabilidade nas curvas, possivelmente devido à baixa representatividade dessas categorias no conjunto de dados. Por outro lado, as classes avião e helicóptero foram identificadas com maior consistência, resultado de uma quantidade maior de exemplos e características visuais mais definidas, facilitando o aprendizado do modelo.

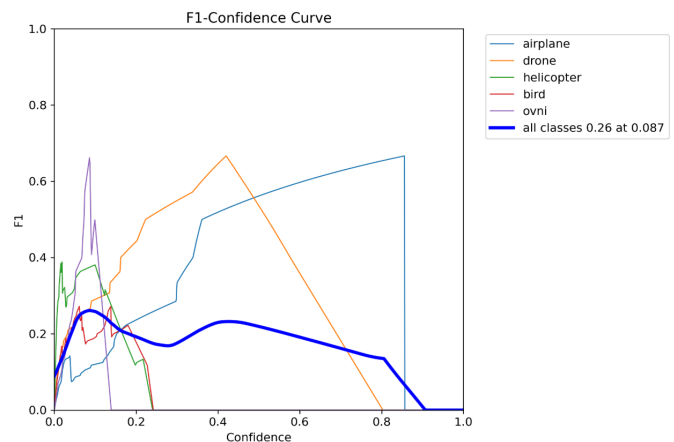


Fig. 5. Resultados da detecção ao longo do tempo.

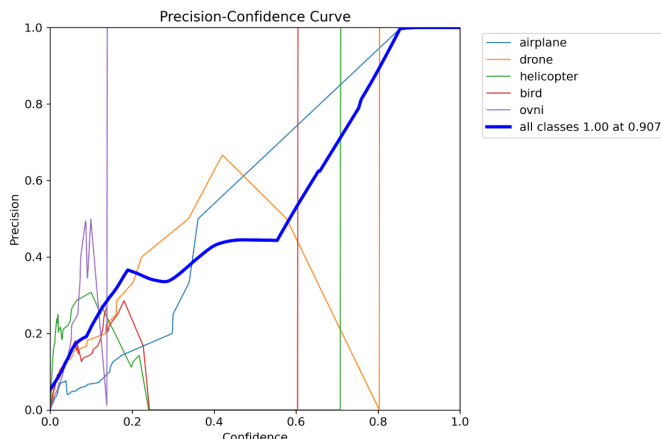


Fig. 6. Precisão média do modelo.

A análise das curvas F1 e precisão em função da confiança indica desempenho limitado e baixa estabilidade do modelo, com pico de F1 (0,26) em baixa confiança e precisão elevada apenas em detrimento do recall. Observa-se confusão entre classes visualmente semelhantes, como pássaro e avião, ou helicóptero e drone, agravada pela escassez de amostras no treinamento. Esses resultados reforçam a necessidade de ampliar e balancear o conjunto de dados para melhorar a confiabilidade e reduzir ambiguidades na classificação.

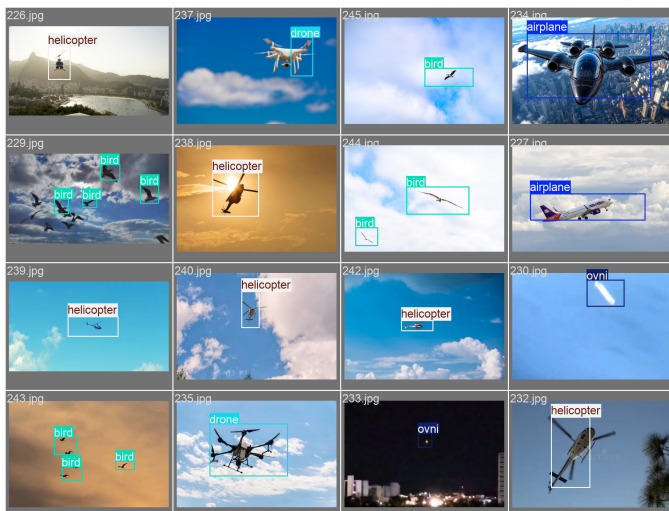


Fig. 7. Labels originais no conjunto de validação.



Fig. 8. Labels geradas pelo algoritmo.

H. Considerações

Após o treinamento, o modelo foi testado em vídeos reais com objetos aéreos. Inicialmente, observou-se instabilidade nas detecções e baixa confiança em alguns quadros, dificultando a identificação precisa dos objetos. Apesar dessas limitações, o sistema conseguiu realizar classificações em tempo real, exibindo rótulos e probabilidades por quadro. Essa etapa reforçou a importância de técnicas como aumento de dados e treinamento adequado para melhorar a robustez e a consistência do modelo em cenários dinâmicos.

III. CONCLUSÃO

Neste trabalho, foi desenvolvido um sistema de detecção automática de objetos aéreos utilizando visão computacional e o modelo YOLOv8, com foco em OVNI, aviões, helicópteros, pássaros e drones. O projeto demonstrou que é possível treinar um modelo leve e customizado mesmo com um conjunto de dados reduzido, empregando técnicas de aumento de dados e imagens compactas para garantir eficiência computacional.

Durante os testes, o modelo mostrou capacidade de detectar e classificar objetos em tempo real, embora tenha apresentado instabilidade inicial em vídeos reais. A ativação do parâmetro `augment=true` foi essencial para melhorar a robustez e a consistência das detecções ao longo dos quadros.

CÓDIGO-FONTE E REPOSITÓRIO

O código-fonte completo deste projeto está disponível online em: <https://github.com/jpedrou/flydetector>.

REFERENCES

- [1] Ultralytics, “Data Augmentation using Ultralytics YOLO”, [Online]. Available: <https://docs.ultralytics.com/guides/yolo-data-augmentation/>. [Accessed: October 28, 2025].
- [2] Ultralytics, “YOLO Documentation,” [Online]. Available: <https://docs.ultralytics.com/>. [Accessed: October 28, 2025].
- [3] Google Images, “Imagens utilizadas no experimento”, [Online]. Available: <https://www.google.com/imghp>. [Accessed: October 28, 2025].